

AUTONOMOUS STRUCTURAL SCREENING

TwinVerse Inspect AI

Inspect the unreachable.

Drone, CCTV and handheld imagery in — located defects, ranked severity, and a shareable report out.

● **Ayaan Aatif**

Team Lead

● **Muhammad Muneeb**

Backend, Data & API

● **Inshrah Mehmood**

Marketing & Outreach

THE PROBLEM

Inspection today is manual, slow, and genuinely dangerous.

● Dangerous

Engineers rappel down dams and climb bridge soffits to look at concrete with their own eyes.

● Slow

A single span can take days to survey. Backlogs mean structures go years between inspections.

● Inconsistent

Two inspectors, two clipboards, two different answers about the same crack.

● Reactive

Defects are found after they matter, not before. Maintenance is repair, not prevention.

WHAT WE BUILT

A first pass that never has to climb anything.

01

Capture

Drone, CCTV, robot or a phone. Images or video.



02

Detect

A fine-tuned YOLOv11 model locates cracks and draws a box around each one.



03

Score

Every detection gets a severity score from a formula shown on screen.



04

Report

Annotated images, a 3D view, and a PDF anyone can forward.

The engineer still decides. The system removes the part of the job that is climbing, photographing, and sorting.

STATUS

Not a mockup. A running system.

104

automated tests

23

endpoints

6.1s

to analyse 8 images

7ms

inference per image

- Upload API with S3-compatible object storage
- Severity engine with a formula served live to the UI
- PDF reports with a limitations page
- Fine-tuned crack detector, trained on 1,317 images
- Next.js dashboard with 3D digital twin viewer
- JWT auth, three roles, Docker Compose, CI

Every number can be recomputed by hand.

severity = area × confidence × class weight

A real detection from the demo data:

0.0288 × 0.558 × 1.0 = 0.01606

The formula, the weights and the band thresholds are served by the API and rendered live — so what the dashboard shows can never drift from what the server computed.

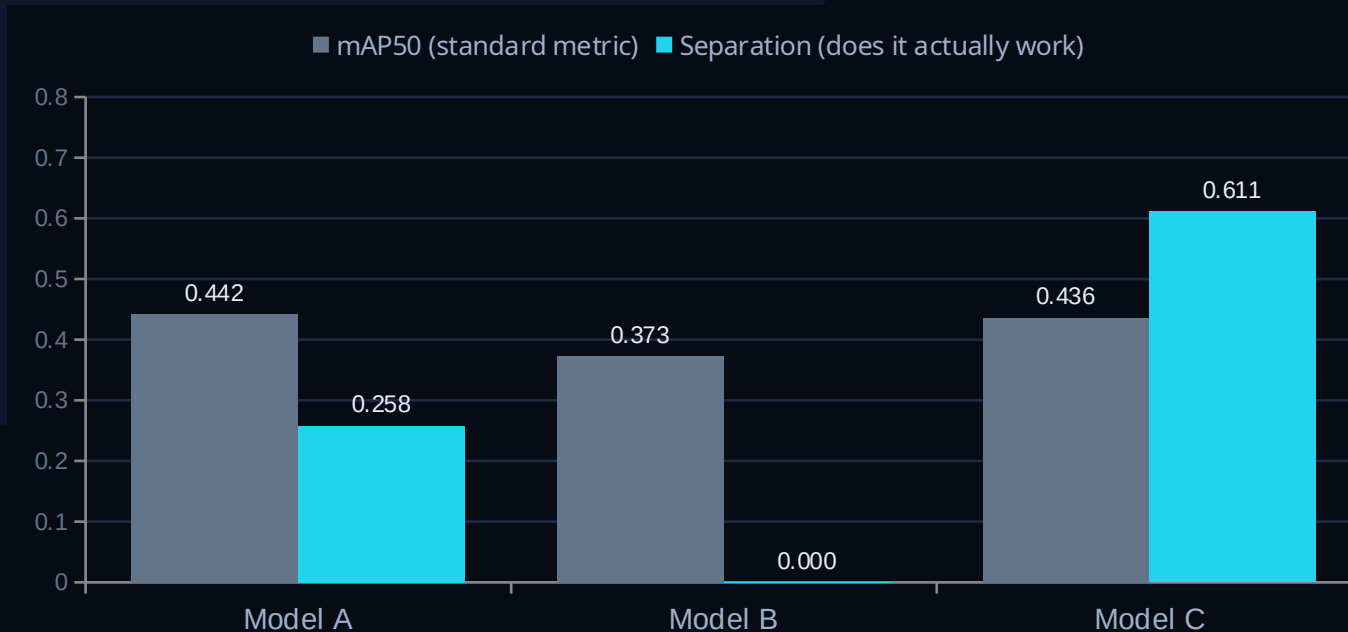
SEVERITY BANDS

- **Critical** > 0.014
- **High** 0.011 – 0.014
- **Medium** 0.009 – 0.011
- **Low** < 0.009

Calibrated against 308 real detections, not assumed.

THE MEASUREMENT THAT MATTERED

The standard metric said all three models were the same. They were not.



What we learned

mAP is computed only over defects that are labelled. A validation set with no clean images cannot catch a model that fires on everything.

So we tested each model against 94 defect-free photographs.

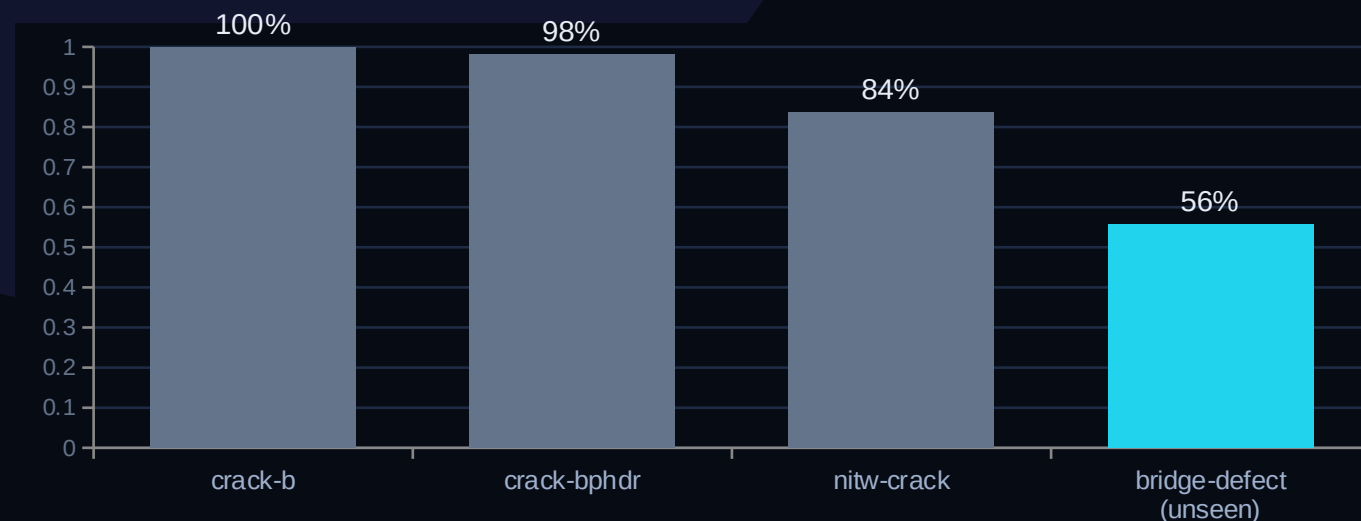
Model A looked fine and flagged half of all clean concrete. Model C flags one in five.

Then we asked the harder question: does it hold up on imagery from somewhere else?

THE TRADE-OFF MOST TEAMS NEVER PRICE

We picked the threshold by simulating an inspection, not by tuning a metric.

A missed crack is found at the next survey, or when something fails. A false alarm costs an engineer thirty seconds. So we stopped optimising a metric that treats them as equal, simulated 500-photograph surveys, and chose the threshold from a recall target.



What this means

Worst case across four datasets: 84% of cracks found. The previous model could not reach 80% on crack-b at any threshold — its best was 35%.

The cost is that we flag 60% of photographs for review. A manual survey means looking at all of them, after climbing the structure.

Recall is a safety decision. Review time is a budget. We priced both.

VISUALISATION

From a photograph to a structure you can orbit.

On the image

- Bounding boxes drawn over each detection
- Colour and size both carry severity
- Hover a finding to see its arithmetic
- Boxes stored normalised, so they align at any zoom

Digital Twin v1

- A 3D structure with glowing severity markers
- Click any marker for its source image and score
- Built in code — no scanning rig required
- Illustrative placement, and we say so on screen

at inspection

NTIW

Riverside Viaduct · Sector 7, North Span · Completed

[Download PDF report](#)

VALIDATION NOBODY ELSE CAN RUN

We built a bridge where we already know where every crack is.

A real crack photograph does not record how far away the camera was, or where the crack is in space. So we generated a bridge in Gazebo with cracks cut from real photographs and placed at known 3D positions, and flew a simulated drone over it — into the same upload endpoint a real drone would use.

1.2 - 3.5 m

Working standoff envelope

Measured, not guessed. Beyond 5 m a crack is too few pixels; at 0.8 m it overflows the frame and returns nothing.

60%

Survey recall on known cracks

11 frames, 5 cracks photographed, 3 found — scored against positions the world knows exactly.

0.71 px

Camera calibration error

Projection convention measured from rendered markers, not derived. The derivation was wrong twice.

The two cracks it missed were the largest and the best-centred. Size does not predict detectability — contrast does.

Play sim_survey.mp4 here — the drone's view with detections and known cracks side by side.

WHAT IT DOES NOT DO

We measured the weaknesses too.

Every one of these is stated in the product itself and on the last page of every report.

Cracks only

One defect class. It has never seen corrosion or spalling and will not report them.

We flag 60% of photographs

Deliberate. Recall is the safety metric; review time is the cost. The threshold is a slider.

56% on the one unseen source

Versus 84-100% on the three in training. Only bridge-defect is wholly unseen, and we say so.

Severity is relative

It ranks which crack to look at first. It does not measure width in millimetres.

The 3D view is illustrative

Marker positions come from capture order, not surveyed coordinates.

Video counts are inflated

Frames are analysed independently, so one crack can be counted many times.

Not an engineering assessment

It is a screening pass. A qualified engineer still signs off.

Naming a weakness costs far less than being caught by it.

UNDER THE HOOD

Boring technology, deliberately.

● Frontend

Next.js 16 · React 19 · Three.js · Tailwind

● API

FastAPI · Python 3.12 · SQLAlchemy · Alembic

● Model

YOLOv11n fine-tuned · PyTorch 2.13 · CUDA 13

● Data

PostgreSQL 16 · MinIO (S3-compatible)

● Ship

Docker Compose · GitHub Actions CI · JWT + RBAC

Decisions on record

21 architectural decisions are written down with their reasoning — including the ones we got wrong, and one we had to publicly reverse after re-checking it.

A dataset that looked ideal and turned out unusable. A merge that made the model worse. A metric that hid a failure for an hour.

All of it is in the repository.

THE TEAM

Three people, one build.



Ayaan Aatif

Team Lead

Architecture, ML pipeline, demo delivery



Muhammad Muneeb

Backend, Data & API

Services, database and the upload pipeline



Inshrah Mehmood

Marketing & Outreach

Positioning, launch material and the pitch

[Add contact email · GitHub · LinkedIn]

WHAT COMES NEXT

The honest roadmap.

SHIPPED

- Crack detection, measured on unseen data
- Severity scoring and PDF reports
- Simulated survey validated on known ground truth

NEXT

- Varied training data to close the 63/81 gap
- More defect classes — corrosion, spalling
- Cross-frame tracking so video counts are real

LATER

- Photogrammetric twin from the imagery itself
- Predictive maintenance once trend data exists
- Live drone and robot integration

Predictive maintenance needs longitudinal data that does not exist yet. We are not going to pretend otherwise.

This does not replace an engineer.

It does the first pass, so the engineer spends their time on judgement instead of data collection — and it tells you exactly how much to trust it.

TwinVerse Inspect AI

Ayaan Aatif · Muhammad Muneeb · Inshrah Mehmood

[Add repository link · contact email]